Report Computational Workflows

Pia Trübenbach and Hanna Lohmann

October 2026

1 Abstract

RNA-sequencing (RNA-seq) measures gene expression and plays an important role in biomedical research. One problem is that pipelines like nf-core/rnaseq often need a lot of memory and computational power. This is a challenge for students using standard laptops. Our goal was to build a lightweight nf-core pipeline that runs at laptops with limited resources.
Therefore we set resource limits, added an optional subsampling step, used the HISAT2 alinger and indexing without splice sites, which needs much less memory. In general the pipeline performs a quality control, trimming, alignment, duplicate marking and gene counting step.
Due to the fact that the lohtrue/analysisrnaseq pipepline is build for educatinal use, we decide to produce plots as one output which visualize the results and make them easier to interpret. The pipeline creates a transcript per million (TPM) table and 3 plots, which show the top 20 expressed genes, the expression distribution per sample and the correlation between samples. In addition, the pipeline outputs a MultiQC report, which contains all quality control results. To make the results reproducible fixed seeds and containers are used.

2 Introduction

In biomedical research due to technological advances it is possible to generate a high amount of biological data. Therefore, the data collection is less of a challenge than the reliable processing, integration and interpretation of this huge amount of data.
One example for this is RNA sequencing (RNA-seq). RNA-seq is a next-generation sequencing technique used to profile the transcriptome, allowing researchers to identify and measure RNA transcripts in a sample at a specific time point. It enables researchers to look for example at differences in gene expression in different groups, post-transcriptional modification and alternative gene spliced transcripts. [35] However, to get some of these meaningful biological insights the raw sequencing data must be computationally processed and analysed.

To process this massive amount of data, bioinformatic pipelines are used. In general, data pipelines are series of automated steps that move information from different sources and formats to a final destination [18]. For RNA-seq data, these pipelines automatically excecute essential processing steps like quality control and read alignment. In Nextflow, a pipeline is made of individual steps that take in data, process it and output the results. These steps are connected by channels that pass the data from from one step to the next one. Because these steps depend on data from the earlier steps in the process, the workflow automatically knows when to run each part without manually scheduling. [31]
Instead of building these pipelines from scratch, researchers can use nf-core. It is a community project which offers a collection of high-quality, pre-built Nextflow pipelines that everyone can use. To simplify development and maintain consistency, nf-core provides standardized templates and tools for developer, while ensuring easy applicable, high quality pipelines for the user. [8] By using these standardized templates and containerized architecture, every analysis becomes fully reproducible. Containers like Docker bundle the pipeline code together with all dependencies and libraries. This ensures that the pipeline runs in an identical environment on any machine leading to reliable and consistent results.
Due to this concept, nf-core ensures that the pipelines follow the FAIR principles.The pipelines are easy to find because they are registered on the central nf-core platform (Findable). They are free accessible for everyone (Accessible), use standard file formats that work across different tools (Interoperable) and include a clear documentation and licenses (Reusable). [36, 8]

Pipelines like nf-core/rnaseq are very powerful and comprehensive, but need a lot of computational resources, making them difficult to run on every laptop. With lohtrue/analysisrnaseq, we tried to create a lightweight nf-core RNA-seq pipeline that easily runs on standard laptops and is suitable for educational use.

3 Methods

The lohtrue/analysisrnaseq pipeline was created with the nf-core template (`nf-core pipelines create`, nf-core/tools version 4.1.0). The template can be created as an official nf-core pipeline which is published by the nf-core community or as a custom pipeline under own GitHub organisation. We chose a custom pipeline under our own organisation (lohtrue), because this allowed us to skip template features that are not needed for a lightweight laptop pipeline, while still using the nf-core structure, modules and tools.[5]

3.1 Pipeline setup and structure

The nf-core template provides a fixed folder structure, so that all nf-core pipelines are organised in the same way. The main file of the pipeline is main.nf, which is executed when the pipeline runs. After initialisation, validation of the parameters and reading of the samplesheet, it runs the main workflow, which is defined in workflows/analysisrnaseq.nf. In this file, all modules are included and connected to define the main workflow. [12]
The most important configuration file is nextflow.config. It contains the default parameters of the pipeline, general Nextflow settings and other information such as the pipeline version. In addition, it defines several profiles, e.g. test profiles, that can be selected when running the pipeline. Further important configuration files are conf/base.config and conf/modules.config. base.config defines default resources for each process and modules.config contains module-specific settings such as tool arguments. [12]
Before running the pipeline, a samplesheet with information about the samples has to be created as a comma-separated file with a header row and passed to the pipeline with the `--input` parameter. The parameters and the samplesheet format are described in nextflow_schema.json and assets/schema_input.json. [12]

3.2 Modules

One of the core concepts of nf-core are modules. A nf-core module is a small, self-contained Nextflow process performing one single, standardized task like running a certain tool. Using nf-core modules has several advantages. Existing modules can be reused and because all modules follow the same structure, they can be easily combined. In addition, each module is documented and already covers all inputs and outputs of a tool. Therefore, less changes are needed later.
nf-core modules are installed with `nf-core modules install modul`. The command copies them into modules/nf-core/ and documents their version in modules.json. To use a module, it is imported in the workflow file with an include statement and called with the input channel. The settings of a module are adjusted in conf/modules.config. [16, 7, 6]
If no suitable nf-core module exists, a local module can be written instead. This can be done by hand or with `nf-core modules create`, which generates a module from the nf-core module template [4]. We wrote two local modules by hand for the last steps. They are stored in modules/local/. Like nf-core modules, each contains one process with inputs, outputs, a script and a container. The actual code is written in Python scripts in bin/. `TPM_TABLE` combines the counts of all samples into one TPM table, and `PLOT_TPM` creates plots from this table using pandas, NumPy and matplotlib (version 3.11.2) [27, 19, 21].
In total, our pipeline uses 13 nf-core modules and our two local modules, which are all listed in Table 1 in the order in which they run. Modules were chosen based on lowest use of resources. For each module, the version of the tool used and the function are given.

Table 1: Modules used in the lohtrue/analysisrnaseq pipeline

Module	Tool (version)	Function
nf-core modules		
`fq/subsample`	fq (0.12.0) [32]	optional subsampling of reads
`fastqc`	FastQC (0.12.1) [1]	quality control of raw reads
`fastp`	fastp (1.3.6) [2]	adapter and quality trimming
`gunzip`	gzip (1.13)	decompression of FASTA and GTF
`hisat2/extractsplicesites`	HISAT2 (2.2.3) [22]	extraction of splice sites from the GTF
`hisat2/build`	HISAT2 (2.2.3) [22]	building the genome index
`hisat2/align`	HISAT2 (2.2.3) [22] , SAMtools (1.24) [24]	alignment of reads, output as BAM
`samtools/fixmate`	SAMtools (1.24) [24]	adding mate information
`samtools/sort`	SAMtools (1.24) [24]	sorting by genomic position
`samtools/markdup`	SAMtools (1.24) [24]	marking duplicate reads
`samtools/flagstat`	SAMtools (1.24) [24]	alignment statistics
`subread/featurecounts`	Subread (2.1.1) [25]	counting reads per gene
`multiqc`	MultiQC (1.35) [15]	summary report
Local modules		
`TPM_TABLE`	Python (3.14.5)	TPM table of all samples
`PLOT_TPM`	Python, pandas, NumPy, matplotlib	plots of TPM values

3.3 Reproducibility and testing

The pipeline was run locally on laptops using Nextflow 26.04.6 [13] and Docker [28]. Each module runs in a fixed container [34], so that the same tool versions are used on every machine. Random steps use fixed seeds. The versions of all tools are recorded in `pipeline_info/` and in the MultiQC report. The code is available on GitHub under the MIT license (release 1.0.0) [33].
For testing, two profiles are included in the repository. The profile `test` runs the whole pipeline on a small dataset, which covers chromosome 19 of the mouse genome (GRCm38, Ensembl release 102) [3, 20] and the test reads are the first 10,000 read pairs of the SRA run SRR23195516 [29], which were provided by the course instructors and had been downloaded with the nf-core/fetchngs pipeline [9]. These reads are also used as a single-end sample to test both input types. Because of 10 % subsampling and limits of 4 CPU and 6 GB memory, the test can be run on a standard laptop. `test_full` uses a larger yeast RNA-seq dataset (GEO GSE110004) from the nf-core test-datasets repository [11].

3.4 Documentation

Basic information and the usage of the pipeline are provided in `README.md`, while `docs/usage.md` and `docs/output.md` describe the input, the parameters and the output files. All tools used are cited in `CITATIONS.md`, and a help message with all parameters, which is generated by `nextflow_schema.json`, can be printed with `--help` [12].

4 Results

4.1 Pipeline

Read preprocessing

Figure 1 in the appendix A shows the structure of the designed RNA-seq pipeline. The input of the pipeline is a samplesheet with sample ID, single- or paired-end FASTQ files and the strandedness (forward, reverse or unstranded) of each sample. As an optional first step, the reads can be subsampled using the nf-core module `fq subsample`. This step only runs if the parameter `--subsample` is set in the run command, otherwise all reads are used. This step reduces the amount of data, so that the pipeline needs less memory and runs faster if computational resources are limited. A fixed seed makes sure that the same reads are selected in every run, so the results are reproducible.
Then `FastQC` checks the quality of the raw reads and removes adapter sequences from the reads and filters out out reads with low quality using `fastp`.

Reference preparation

If the reference genome (FASTA) or the annotation (GTF) file is compressed, it is first decompressed with `gunzip`. As an aligner we chose `--HISAT2` because it needs less memory than other aligners such as `--STAR`. In addition, the GTF file with the gene and exon positions is passed directly to `featureCounts`, which uses it later to count the reads per gene.
`HISAT2_extractsplicesites` extracts the known splice sites from the GTF file. After that `HISAT2_build` builds an index of the refernce genome. In the pipeline the index is built without splice sites to save memory. This is achieved by using a memory threshold in the nf-core module `HISAT2_BUILD`. The module only uses the splice sites if the memory available for the task is higher than the threshold, which we set to 200 GB. By default, each task gets at most 6 GB of memory (`--max_memory`) and even without this limit `HISAT2_BUILD` would get only 72 GB. Because this value is always below the threshhold, the index is always built without splice sites. The splice sites are given to the alignment step instead.

Alignment and post-processing

`HISAT2_align` aligns the reads to the reference genome. It uses the strandedness from the samplesheet, the extracted splice sites and a fixed seed (seed 22). Using a seed makes sure that reads which map equally well to several positions are every time assigned in the same way, so the alignment maintain reproducible. For paired-end reads, `samtools fixmate` adds information about the mate of each read, which is required for correct marking of the duplicates. After that, `samtools sort` sorts the alignments based on their position and `samtools markdup` marks duplicate reads. Then `samtools flagstat` counts how many reads were mapped to the genome, how many read pairs were mapped correctly and how many reads are duplicates. From these values one can conclude if the alignment worked well and they are also listed in the final MultiQC report.

Quantification and reporting

The modul `featureCounts` counts how many reads relate to each gene. For this, it makes use of the exon positions stored in the GTF file and the strandedness of each sample.
The local module `tpm_table` then transform these counts into transcripts per million (TPM) by first correcting the counts for gene length. After that, these corrected values of all genes in a sample are scaled, meaning that they add up to one million. This normalization allows a comparison of the expressiuon levels of genes within a sample and between samples. The result is a table in TSV format with one row for each gene and one column for each sample. The local module `plot_tpm` uses the TPM table to create 3 plots (png format), which show the top 20 expressed genes, the expression distribution per sample and the correlation between samples. In addition, the pipeline outputs a final MultiQC report (HTML format), which collects the results of FastQC, fastp, HISAT2, samtools flagstat and featureCounts, together with the TPM plots.

4.2 Performance

Nextflow can run the same step for different samples at the same time. To see the effect of this, we ran our pipeline once with one sample and once with three samples and compared the processing time (all steps with and without MultiQC, Table 2). MultiQC was not included in the processing time, because it runs only once at the end for all samples. Therefore, it does not show the effect of parallel processing between samples. With one sample, the processing took 8.0 s, with three samples 16.7 s. So three times more samples needed only about twice the time, because the samples were processed in parallel.

Table 2: Runtime of lohtrue/analysisrnaseq pipepline with one and three samples.

	1 sample	**3 samples**
Number of tasks	13	29
Total runtime	26.0 s	47.7 s
Processing time without MultiQC	8.0 s	16.7 s

With larger data sets, the effect of parallelisation would probably be bigger.

5 Discussion

5.1 Reproducibility and FAIR principles

nf-core ensures reproducibility through containers for every module, versioned modules, parameter validation with a JSON schema and also automated tests. With every pull request, continuous integration (CI) runs `nf-core pipelines lint` and nf-test, which compares the outputs with a stored snapshot [12, 17]. In addition, nf-core pipelines, which are listed on the nf-core website, use standard file formats and come with documentation and an MIT license, which makes them FAIR [36].
Our pipeline uses containers, fixed seeds and automatic version reporting. It is publicly available under the MIT license and has two test profiles. However, the nf-test snapshot has not been created and therefore the CI testing doesn‘t work yet. As a result changes to the code are not checked automatically. The pipeline was only tested with Docker and not with a Conda environment. Furthermore, the pipeline is not registered in a workflow registry. Setting up nf-test and CI and creating a release with a DOI could therefore be the next steps.

5.2 Limitations and outlook

As lohtrue/analysisrnaseq is a lightweight pipeline for educational use, it is missing some steps of a complete RNA-seq workflow. The strandedness has to be given manually, although a wrong value leads to wrong counts. An automatic detection, as in nf-core/rnaseq using Salmon [10, 30], would make the pipeline safer to use. Moreover, only HISAT2 is available as an aligner. Offering alternative aligners to choose from such as STAR [14] or Salmon would allow users to compare different methods. Furthermore, other RNA-seq specific quality control steps, like the removal of ribosomal RNA with `SortMeRNA` could be introduced to the pipeline [23]. The TPM values from featureCounts are only an approximation. A differential expression analysis, e.g. with DESeq2 [26], would be the next step to answer biological questions. Finally, the pipeline was only tested with small datasets, and all processes request up to 6 GB of memory although they used at most about 1.1 GB. Therefore, adjusting these requests would allow more tasks to run in parallel.

References

[1] Simon Andrews. *FastQC: A Quality Control Tool for High Throughput Sequence Data*. Version 0.12.1. Babraham Bioinformatics. 2010. URL: https://www.bioinformatics.babraham.ac.uk/projects/fastqc/ (visited on 10/18/2026).

[2] Shifu Chen et al. "fastp: an ultra-fast all-in-one FASTQ preprocessor". In: *Bioinformatics* 34.17 (2018), pp. i884–i890. DOI: 10.1093/bioinformatics/bty560.

[3] Deanna M. Church et al. "Lineage-specific biology revealed by a finished genome assembly of the mouse". In: *PLoS Biology* 7.5 (2009), e1000112. DOI: 10.1371/journal.pbio.1000112.

[4] nf-core community. *Create a module – nf-core/tools documentation*. nf-core. URL: https://nf-co.re/docs/nf-core-tools/cli/modules/create (visited on 10/18/2026).

[5] nf-core community. *Creating a new pipeline*. nf-core. URL: https://nf-co.re/docs/nf-core-tools/cli/pipelines/create (visited on 10/18/2026).

[6] nf-core community. *Modules*. nf-core. URL: https://nf-co.re/modules (visited on 10/18/2026).

[7] nf-core community. *nf-core modules install*. nf-core. URL: https://nf-co.re/docs/nf-core-tools/cli/modules/install (visited on 10/18/2026).

[8] nf-core community. *nf-core: A community effort to collect a curated set of analysis pipelines built using Nextflow*. URL: https://nf-co.re/ (visited on 10/16/2026).

[9] nf-core community. *nf-core/fetchngs: Pipeline to fetch metadata and raw FastQ files from public databases*. nf-core. URL: https://nf-co.re/fetchngs (visited on 10/18/2026).

[10] nf-core community. *nf-core/rnaseq: RNA sequencing analysis pipeline*. nf-core. URL: https://nf-co.re/rnaseq (visited on 10/21/2026).

[11] nf-core community. *nf-core/test-datasets, branch rnaseq*. URL: https://github.com/nf-core/test-datasets/tree/rnaseq (visited on 10/18/2026).

[12] nf-core community. *Pipeline file structure*. nf-core. URL: https://nf-co.re/docs/developing/pipelines/template-files (visited on 10/18/2026).

[13] Paolo Di Tommaso et al. "Nextflow enables reproducible computational workflows". In: *Nature Biotechnology* 35.4 (2017), pp. 316–319. DOI: 10.1038/nbt.3820.

[14] Alexander Dobin et al. "STAR: ultrafast universal RNA-seq aligner". In: *Bioinformatics* 29.1 (2013), pp. 15–21. DOI: 10.1093/bioinformatics/bts635.

[15] Philip Ewels et al. "MultiQC: summarize analysis results for multiple tools and samples in a single report". In: *Bioinformatics* 32.19 (2016), pp. 3047–3048. ISSN: 1367-4803. DOI: 10.1093/bioinformatics/btw354. URL: http://dx.doi.org/10.1093/bioinformatics/btw354.

[16] James A. Fellows Yates. *Writing nf-core modules – Chapter 1: Introduction*. nf-core. URL: https://nf-co.re/docs/developing/tutorials/writing-nf-core-modules/1-introduction (visited on 10/18/2026).

[17] Lukas Forer and Sebastian Schönherr. *nf-test: a simple testing framework for Nextflow pipelines*. URL: https://www.nf-test.com (visited on 10/21/2026).

[18] Halfcapsule. *Understanding Data Pipelines: Architecture and Workflow*. Medium. July 19, 2023. URL: https://medium.com/@halfcapsule/understanding-data-pipelines-architecture-and-workflow-4f7ab1e864e6 (visited on 10/16/2026).

[19] Charles R. Harris et al. "Array programming with NumPy". In: *Nature* 585.7825 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[20] Kevin L. Howe et al. "Ensembl 2021". In: *Nucleic Acids Research* 49.D1 (2021), pp. D884–D891. DOI: 10.1093/nar/gkaa942.

[21] John D. Hunter. "Matplotlib: A 2D Graphics Environment". In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. DOI: 10.1109/MCSE.2007.55.

[22] Daehwan Kim et al. "Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype". In: *Nature Biotechnology* 37.8 (2019), pp. 907–915. DOI: 10.1038/s41587-019-0201-4.

[23] Evguenia Kopylova, Laurent Noé, and Hélène Touzet. "SortMeRNA: fast and accurate filtering of ribosomal RNAs in metatranscriptomic data". In: *Bioinformatics* 28.24 (2012), pp. 3211–3217. DOI: 10.1093/bioinformatics/bts611.

[24] Heng Li et al. “The Sequence Alignment/Map format and SAMtools”. In: *Bioinformatics* 25.16 (2009), pp. 2078–2079. DOI: 10.1093/bioinformatics/btp352.

[25] Yang Liao, Gordon K. Smyth, and Wei Shi. “featureCounts: an efficient general purpose program for assigning sequence reads to genomic features”. In: *Bioinformatics* 30.7 (2014), pp. 923–930. DOI: 10.1093/bioinformatics/btt656.

[26] Michael I. Love, Wolfgang Huber, and Simon Anders. “Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2”. In: *Genome Biology* 15.12 (2014), p. 550. DOI: 10.1186/s13059-014-0550-8.

[27] Wes McKinney. “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a.

[28] Dirk Merkel. “Docker: lightweight Linux containers for consistent development and deployment”. In: *Linux Journal* 2014.239 (2014), p. 2.

[29] NCBI Sequence Read Archive. *Run SRR23195516*. URL: https://www.ncbi.nlm.nih.gov/sra/SRR23195516 (visited on 10/18/2026).

[30] Rob Patro et al. “Salmon provides fast and bias-aware quantification of transcript expression”. In: *Nature Methods* 14.4 (2017), pp. 417–419. DOI: 10.1038/nmeth.4197.

[31] Seqera. *Nextflow Documentation: Overview*. Seqera Docs. URL: https://docs.seqera.io/nextflow/overview (visited on 10/16/2026).

[32] St. Jude Rust Labs. *fq: fq filters, subsamples, and validates FASTQ files*. Version 0.12.0. URL: https://github.com/stjude-rust-labs/fq (visited on 10/18/2026).

[33] Pia Trübenbach and Hanna Lohmann. *lohtrue/analysisrnaseq: a lightweight nf-core RNA-seq pipeline*. Version 1.0.0. 2026. URL: https://github.com/lohtrue/analysisrnaseq (visited on 10/18/2026).

[34] Felipe da Veiga Leprevost et al. “BioContainers: an open-source and community-driven framework for software standardization”. In: *Bioinformatics* 33.16 (2017), pp. 2580–2582. DOI: 10.1093/bioinformatics/btx192.

[35] Zhong Wang, Mark Gerstein, and Michael Snyder. “RNA-Seq: a revolutionary tool for transcriptomics”. In: *Nature Reviews Genetics* 10.1 (2009), pp. 57–63. DOI: 10.1038/nrg2484. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC2949280/.

[36] Mark D. Wilkinson et al. “The FAIR Guiding Principles for scientific data management and stewardship”. In: *Scientific Data* 3 (2016), p. 160018. DOI: 10.1038/sdata.2016.18. URL: https://www.nature.com/articles/sdata201618.

A Metro map

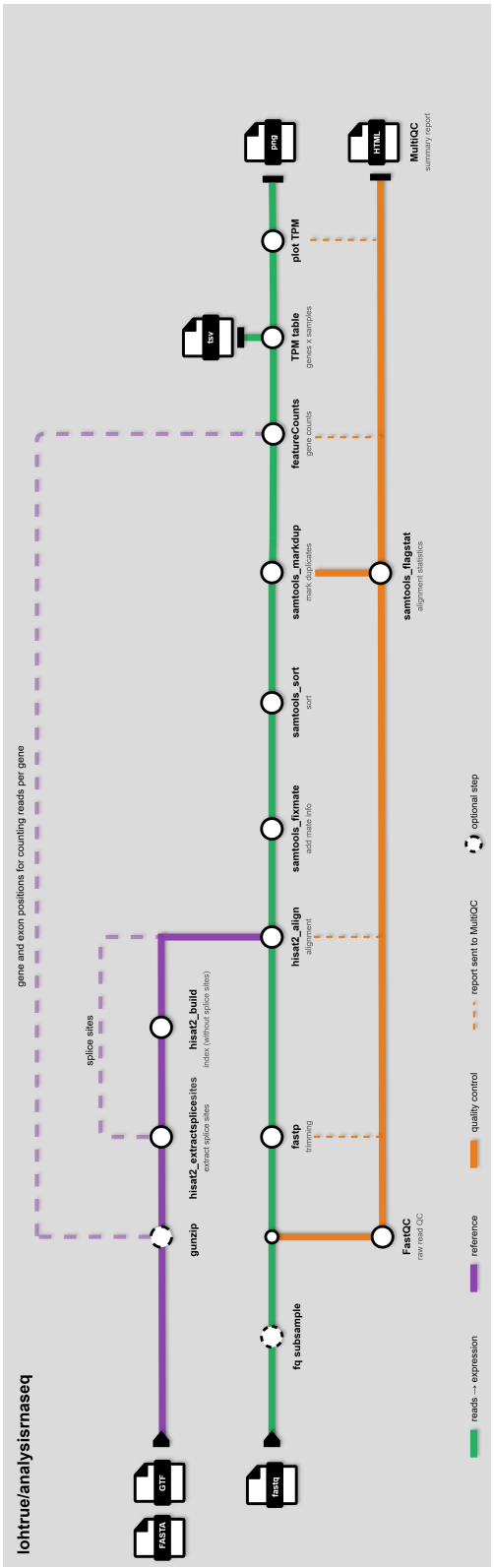


Figure 1: Metro map of the lohtrue/analysisrnaseq pipeline.